Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

Digital Assignment - 1

On

PROJECT TITLE:

" Fast Critical Node Detection in Large-Scale Networks"

Submitted by

PALLAVI YADAV - 22MID0028

ABDUR RAHUMAN – 22MID0044

For

Advanced Graph Algorithms

Slot: A1 +TA1

1. Problem Understanding & Motivation

1.1 Problem Definition

Large-scale real-world systems such as communication networks, power grids, transportation systems, and social networks can be represented as graphs, where nodes represent entities (e.g., routers, substations, users) and edges represent connections or interactions. The overall functionality and connectivity of these networks depend heavily on certain structurally important nodes known as **critical nodes**.

A **critical node** is defined as a node whose removal causes a significant reduction in network connectivity, efficiency, or robustness. The **Critical Node Detection Problem (CNDP)** involves identifying such nodes so that their removal results in maximum fragmentation of the network. This is typically measured using metrics such as reduction in the size of the largest connected component, increase in disconnected components, or decrease in reachability between nodes.

This problem is computationally difficult, especially for large networks, because the number of possible node combinations grows exponentially with network size.

1.2 Relevance and Importance

Critical node detection is essential in many real-world applications:

- **Communication Networks:** Failure of key routers or servers can disrupt Internet connectivity and communication services.
- **Power Grids:** Failure of critical substations may lead to cascading failures and large-scale blackouts.
- **Social Networks:** Identifying influential users helps control misinformation and understand information spread.
- **Transportation Networks:** Failure of major hubs such as airports or railway stations can severely disrupt transportation.
- **Cybersecurity:** Detecting vulnerable nodes helps protect infrastructure from targeted attacks.

In all these systems, identifying critical nodes improves network robustness, fault tolerance, and security.

1.3 Challenges in Large-Scale Networks (100K–1M Nodes)

Modern networks often contain hundreds of thousands to millions of nodes, which introduces major challenges:

- **High computational complexity**, making exact methods impractical
- **Large memory requirements** for storing and processing graph data

- **Slow performance** of traditional algorithms on large datasets
- **Limited scalability**, as most existing methods are tested only up to ~10K nodes

There is a need for fast and scalable algorithms capable of handling large networks efficiently.

1.4 Challenges in Temporal and Evolving Networks

Real-world networks are dynamic and evolve over time. Connections and interaction strengths change continuously, resulting in **temporal networks**.

Key challenges include:

- Changing edge weights over time
- Dynamic changes in node importance
- Need for frequent recalculation of critical nodes
- High computational cost of recomputation

This requires adaptive algorithms that can efficiently update node importance without recomputing everything from scratch.

1.5 Motivation for This Project

The primary motivation of this project is to develop a **fast, scalable, and adaptive critical node detection framework** that can:

- Handle large-scale networks with 100K–1M nodes
- Support temporal and evolving networks
- Efficiently recalculate node importance using adaptive weight updates
- Improve computational efficiency compared to traditional methods
- Enhance robustness analysis for real-world infrastructure

This will help overcome the scalability and adaptability limitations of existing critical node detection approaches.

2. Graph Modeling and Representation

2.1 Introduction

Graph modeling is an effective abstraction technique used to represent complex networks in a structured mathematical form. In this project, large-scale networks such as communication networks, power grids, transportation systems, and social networks are modeled as graphs. This abstraction enables efficient analysis of connectivity, node importance, and network robustness, which is essential for identifying critical nodes.

The graph model captures the structural and functional relationships between entities and allows scalable processing of networks containing up to 100,000–1,000,000 nodes.

2.2 Graph Abstraction Model

The network is formally represented as a weighted temporal graph:

$$G = (V, E, W, T)$$

Where:

- V= set of nodes (vertices)
- E= set of edges (connections between nodes)
- W= set of edge weights
- T= temporal component representing time evolution

This abstraction allows representation of both static and dynamic network behaviour.

2.3 Node Representation

Nodes represent individual entities in the network. Each node corresponds to a physical or logical component depending on the application domain.

Examples include:

Application Domain	**Node Representation**
Communication Network	Router / Server
Power Grid	Substation
Social Network	User
Transportation Network	Airport / Station

Mathematically, the node set is defined as:

$$V = \{v_1, v_2, v_3, \ldots, v_n\}$$

Where:

- $n$represents the total number of nodes
- In this project:

$$10^5 \leq n \leq 10^6$$

Each node may also store additional attributes such as degree, connectivity score, or centrality value.

2.4 Edge Representation

Edges represent relationships or connections between nodes.

Mathematically:

$$E = \{(u, v) \mid u, v \in V\}$$

Edges define how nodes interact within the network.

Edges may be classified as:

- **Undirected edges:** Connection exists in both directions
- **Directed edges:** Connection exists in one direction

For example:

- Undirected: social friendships
- Directed: communication flow

Edges define the network topology and connectivity structure.

2.5 Edge Weight Representation

Edge weights represent the strength, capacity, or importance of the connection between nodes.

$$W(u, v) = w_{uv}$$

Where:

- w_{uv}represents the weight of the edge between nodes $u$and v

Edge weights may represent:

Network Type	**Weight Meaning**
Communication network	Traffic load
Power grid	Power transmission capacity
Social network	Interaction frequency
Transportation network	Distance or travel time

Weighted graphs provide a more realistic representation of real-world networks.

2.6 Temporal Graph Representation

Since real-world networks evolve over time, the graph is modeled as a temporal graph:

$$G_t = (V_t, E_t, W_t)$$

Where:

- t represents time
- Edge connections and weights may change dynamically

This allows adaptive recalculation of node importance based on changing network conditions.

Weight update model:

$$W_{new}(u, v) = \alpha W_{old}(u, v) + (1 - \alpha) W_{current}(u, v)$$

Where:

- $\alpha$is the weight decay factor
- Ensures smooth temporal adaptation

2.7 Graph Storage Representation

To support large-scale networks efficiently, the graph is stored using an **Adjacency List Representation**.

In this representation:

- Each node stores a list of its neighbouring nodes
- Only existing edges are stored

Memory complexity:

$$O(|V| + |E|)$$

This is highly efficient compared to adjacency matrix representation, which requires:

$$O(|V|^2)$$

Adjacency lists are suitable for sparse networks with millions of nodes.

2.8 Connectivity Modeling

Connectivity defines the reachability between nodes.

Important connectivity metrics include:

- Largest Connected Component (LCC)
- Number of connected components
- Reachability between nodes

If removal of a node significantly reduces connectivity, it is classified as a critical node.

2.9 Constraints and Scalability Considerations

The graph model must satisfy the following constraints:

1. Large-Scale Constraint

$$|V| = 100{,}000 \text{ to } 1{,}000{,}000$$
$$|E| = 1{,}000{,}000 \text{ to } 10{,}000{,}000$$

The graph structure must support efficient processing at this scale.

2. Memory Constraint

Efficient storage mechanisms such as adjacency lists are required to minimize memory usage.

3. Computational Constraint

Graph operations such as traversal and connectivity computation must be efficient.

4. Temporal Constraint

The graph must support dynamic updates such as:

- Edge weight changes
- Edge addition or removal
- Adaptive recalculation

2.10 Critical Node Representation in Graph Model

Let:

$$C \subseteq V$$

represent the set of critical nodes.

After removal:

$$G' = G - C$$

The resulting graph G'has reduced connectivity.

The objective is to identify nodes whose removal produces maximum network fragmentation.

3. Algorithm Selection and Justification

3.1 Introduction

The Critical Node Detection Problem (CNDP) is computationally complex, especially for large-scale networks containing 100,000 to 1,000,000 nodes. Exact algorithms are impractical due to their exponential time complexity. Therefore, this project adopts scalable graph algorithms that provide efficient and accurate approximation of critical nodes while maintaining computational feasibility.

The selected algorithms are based on graph connectivity analysis, centrality measures, and scalable heuristic approaches. These algorithms are chosen due to their efficiency, scalability, and suitability for temporal and large-scale network environments.

3.2 Algorithm Selection Criteria

The algorithms were selected based on the following criteria:

1. Scalability to large networks (100K–1M nodes)
2. Low computational complexity
3. Ability to handle weighted and temporal graphs
4. Accuracy in identifying critical nodes
5. Efficient memory utilization
6. Capability for adaptive updates in evolving networks

These criteria ensure the algorithms are suitable for real-world large-scale network analysis.

3.3 Selected Algorithms Overview

The following algorithms are selected for implementation:

1. Degree Centrality Algorithm
2. Betweenness Centrality Algorithm (Approximate Version)
3. Greedy Critical Node Removal Algorithm
4. Connected Component Analysis using Breadth-First Search (BFS)
5. Adaptive Weight Update Algorithm for Temporal Networks

Each algorithm plays a specific role in identifying and validating critical nodes.

3.4 Degree Centrality Algorithm

Definition

Degree centrality measures the number of direct connections a node has.

Mathematically:

$$DC(v) = deg(v)$$

Where:

- $deg(v)$= number of edges connected to node v

Justification

Nodes with high degree centrality are highly connected and play an important role in maintaining network connectivity.

Removal of such nodes results in significant fragmentation.

Advantages

- Very fast computation
- Time complexity:

$$O(V + E)$$

- Suitable for large-scale networks
- Easy to update dynamically

Suitability

Highly suitable as a baseline algorithm for large-scale critical node detection.

3.5 Betweenness Centrality Algorithm (Approximate)

Definition

Betweenness centrality measures how often a node appears on shortest paths between other nodes.

Mathematically:

$$BC(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

Where:

- σ_{st}= total number of shortest paths between nodes $s$and t

- $\sigma_{st}(v)$= number of shortest paths passing through node v

Justification

Nodes with high betweenness centrality act as bridges connecting different network regions.

Removal of these nodes causes maximum connectivity disruption.

Advantages

- Accurately identifies structurally important nodes
- Effective for fragmentation analysis

Limitation

Exact computation complexity:

$$O(V \times E)$$

Therefore, approximate methods are used for scalability.

Suitability

Suitable for identifying bridge nodes in large networks using approximation methods.

3.6 Greedy Critical Node Removal Algorithm

Definition

This algorithm iteratively removes the node that causes maximum connectivity reduction.

Steps:

1. Compute importance score for all nodes
2. Remove the node with highest score
3. Recalculate connectivity
4. Repeat until required number of nodes are removed

Justification

This approach directly optimizes network fragmentation.

It is more effective than single-step centrality methods.

Advantages

- High accuracy
- Efficient approximation

- Scalable to large networks

Time complexity:

$$O(k \times (V + E))$$

Where:

- k= number of nodes to remove

Suitability

Highly suitable for scalable critical node detection.

3.7 Connectivity Analysis using Breadth-First Search (BFS)

Definition

Breadth-First Search is used to compute connected components.

Time complexity:

$$O(V + E)$$

Purpose

Used to measure network connectivity after node removal.

Metrics computed:

- Number of connected components
- Size of largest connected component

Justification

Efficient and scalable for large networks.

Suitability

Essential for validating impact of node removal.

3.8 Adaptive Weight Update Algorithm

Definition

This algorithm updates edge weights dynamically in temporal networks.

Weight update formula:

$$W_{new}(u, v) = \alpha W_{old}(u, v) + (1 - \alpha) W_{current}(u, v)$$

Where:

- α= decay factor

Justification

Allows network to adapt to changing conditions.

Ensures accurate node importance calculation.

Advantages

- Supports temporal networks
- Enables dynamic analysis
- Reduces need for full recomputation

Suitability

Critical for evolving network environments.

3.9 Combined Algorithm Framework

The final system combines all algorithms:

Step 1: Represent graph using adjacency list
Step 2: Compute node importance using centrality algorithms
Step 3: Remove candidate critical node using greedy approach
Step 4: Recalculate connectivity using BFS
Step 5: Update weights using adaptive update model
Step 6: Repeat for temporal updates

This integrated approach ensures accuracy and scalability.

3.10 Correctness Justification

The correctness of the selected algorithms is justified as follows:

- Degree centrality correctly identifies highly connected nodes
- Betweenness centrality correctly identifies bridge nodes
- Greedy removal directly optimizes connectivity reduction
- BFS accurately computes connectivity metrics
- Adaptive update ensures temporal correctness

These algorithms are based on well-established graph theory principles.

3.11 Scalability Justification

The selected algorithms are scalable because:

- Time complexity is linear or near-linear
- Adjacency list representation reduces memory usage
- Approximate centrality reduces computational cost
- Greedy algorithm provides efficient optimization

This ensures applicability to networks with up to 1,000,000 nodes.

4. Depth of Background Study

4.1 Overview of the Critical Node Detection Problem

The Critical Node Detection Problem (CNDP) is a well-established problem in graph theory and network science, focusing on identifying nodes whose removal significantly reduces network connectivity, efficiency, or robustness. This problem is particularly important in applications such as communication networks, power grids, transportation systems, and social networks, where node failures can disrupt system functionality.

CNDP is considered an NP-hard combinatorial optimization problem, meaning that finding an exact optimal solution becomes computationally infeasible for large networks. Consequently, researchers have developed several alternative approaches that balance accuracy and computational efficiency. These approaches can be broadly categorized into exact methods, centrality-based methods, heuristic algorithms, community-based approaches, metaheuristic optimization, approximation techniques, and recent machine learning-based methods.

4.2 Exact Optimization Methods

Exact methods attempt to identify the optimal set of critical nodes by evaluating all possible combinations or by solving mathematical optimization models such as Integer Linear Programming (ILP).

These methods guarantee optimal results but suffer from extremely high computational complexity. The number of possible node combinations increases exponentially with network size, making exact methods impractical for networks containing more than a few thousand nodes. Additionally, exact optimization techniques require significant computational time and memory, which limits their applicability to small-scale networks.

Therefore, while exact methods provide theoretical understanding, they are not suitable for large-scale real-world networks.

4.3 Centrality-Based Approaches

Centrality-based methods identify critical nodes based on their structural importance within the network. These methods are widely used due to their simplicity and scalability.

Degree centrality is one of the simplest measures, which identifies nodes with the highest number of direct connections. Nodes with high degree values are considered important because their removal can affect many neighboring nodes. However, degree centrality considers only local connectivity and may not accurately capture the global importance of nodes.

Betweenness centrality measures how frequently a node appears on the shortest paths between other nodes. Nodes with high betweenness centrality act as bridges connecting different regions of the network. Removing such nodes can significantly fragment the network. Although betweenness centrality provides accurate identification of critical nodes, its computation is expensive for large networks. Approximation techniques are often used to improve scalability.

Closeness centrality measures how close a node is to all other nodes in the network. Nodes with high closeness centrality can efficiently communicate with other nodes and play an important role in maintaining network efficiency. However, this method also requires significant computational effort for large networks.

Overall, centrality-based methods provide efficient and scalable solutions but may not always produce optimal results.

4.4 Greedy Heuristic Algorithms

Greedy heuristic algorithms are widely used for critical node detection due to their efficiency and scalability. These algorithms iteratively remove nodes based on their importance and update the network structure after each removal.

In each iteration, the algorithm selects the node whose removal causes the maximum reduction in connectivity. The network structure is then updated, and node importance is recalculated. This process continues until the required number of nodes is identified.

Greedy algorithms provide a good balance between computational efficiency and solution quality. They are significantly faster than exact methods and can be applied to large-scale networks. However, since greedy algorithms rely on heuristic decisions, they do not guarantee optimal solutions.

Despite this limitation, greedy methods are among the most practical approaches for large-scale network analysis.

4.5 Community-Based Approaches

Community-based approaches focus on identifying groups of nodes that are densely connected internally but sparsely connected to other groups. Nodes that connect different communities play a critical role in maintaining overall network connectivity.

By identifying and removing such bridge nodes, community-based methods can effectively fragment the network. These methods are particularly effective in real-world networks that exhibit strong community structures, such as social networks.

However, the effectiveness of these methods depends on the accuracy of the community detection algorithm.

4.6 Metaheuristic Optimization Methods

Metaheuristic methods such as Genetic Algorithms, Particle Swarm Optimization, and Simulated Annealing have been applied to the critical node detection problem. These methods use optimization techniques to search for near-optimal solutions in complex networks.

Metaheuristic methods can provide high-quality solutions but require significant computational time and resources. As a result, they are generally not suitable for very large networks containing hundreds of thousands or millions of nodes.

4.7 Approximation Algorithms

Approximation algorithms provide near-optimal solutions with significantly lower computational cost. These methods use efficient heuristics and sampling techniques to estimate node importance.

Approximation algorithms are scalable and suitable for large networks. Although they may not always produce optimal solutions, they provide sufficiently accurate results for practical applications.

These methods are widely used in large-scale network analysis due to their efficiency.

4.8 Temporal Network Analysis

Most traditional critical node detection methods assume static networks. However, real-world networks are dynamic and evolve over time. Edge connections and weights may change continuously.

Temporal network analysis considers these changes and updates node importance accordingly. However, frequent recalculation increases computational cost and complexity. Existing temporal methods often lack scalability and efficiency for large-scale networks.

This highlights the need for adaptive and scalable approaches for temporal critical node detection.

4.9 Limitations of Existing Approaches

Based on the literature review, the following limitations are identified:

- Exact methods are not scalable for large networks
- Metaheuristic methods have high computational cost
- Centrality-based methods may not always provide optimal results
- Limited support for temporal networks
- Lack of efficient algorithms for networks with millions of nodes

These limitations create a need for scalable and adaptive solutions.

S. No.	Reference (Year)	Method / Contribution	Key Findings
1	Predicting Critical Nodes in Temporal Networks by Dynamic GCNs (2023)	GCN + LSTM framework to identify critical nodes in temporal networks using snapshot embeddings and sequential learning.	Demonstrates improved ranking accuracy and linear scalability with network size. (MDPI)
2	*Critical Nodes Identification in Complex Networks: A Survey* (2025)	Comprehensive survey categorising methods (centrality, dynamic, AI, higher-order).	Highlights gaps in dynamic and large-scale critical node identification frameworks. (arXiv)
3	A Distributed Algorithm for Critical Node Identification in IoT Networks (2025)	Distributed solution for the Component-Cardinality Constrained Critical Node Problem (3C-CNP).	First distributed method for CNDP variant; validated via simulations on IoT networks. (jcoms.fesb.unist.hr)
4	Efficient Algorithms for Critical Node Detection on Path Graphs (2025)	Dynamic programming formulations for CNDP on structured graph classes.	Provides optimal solutions in polynomial time for path graphs under binary connection metrics. (Science Publishing Group)
5	Dangling Centrality: Critical Node Detection via Link Removal (2025)	Introduces "Dangling Centrality" metric to evaluate node importance based on link deletion impact.	Validated on real-world PPI and social networks showing improved node importance rankings. (Nature)
6	Critical Node Detection in Temporal Social Networks (2025)	Proposes new temporal centrality measures (temporal supracycle ratio & semi-local).	Improves identification accuracy under SIR epidemic simulation compared to classical centrality. (PubMed)
7	A Multi-Attribute Decision-Making Approach (2024)	Combines multiple centrality attributes using CRITIC-TOPSIS for node importance.	Better integrates diverse network metrics for critical node ranking. (MDPI)
8	Dynamic Identification of Important Nodes (2024)	KPDN–INCC method focusing on dynamic re-evaluation of node importance.	Demonstrates need for re-assessment during node removal in evolving networks. (Nature)
9	Learning to Detect Critical Nodes in Sparse Graphs (2024)	Feature importance-aware attention network combined with deep RL.	End-to-end graph learning for CNP without problem-specific heuristics. (aimodels.fyi)
10	Seeking Critical Nodes in Digraphs (2023)	Heuristic analysis and CNH software implementation for directed CNDP.	Compares heuristics performance for minimizing pairwise connectivity in digraphs. (sciencedirect.com)

S. No.	Reference (Year)	Method / Contribution	Key Findings
11	Critical Node Detection in Hypergraphs using Weighted Degree (2023)	Extends CNDP to hypergraphs with weighted node degree centrality.	Applies CNDP to higher-order structures showing promising results on real data. (peerj.com)
12	Scalable GNN-based Framework for Critical Node Identification (2020)	GNN model that learns criticality scores on representative subsets for large networks.	Demonstrates scalability advantages over conventional approaches in large graphs. (arXiv)

5. Methodology / System Design

5.1 Overview

The proposed system is designed to efficiently detect critical nodes in large-scale and temporal networks using a scalable graph-based framework. The methodology consists of graph construction, node importance computation, critical node identification, connectivity evaluation, and adaptive weight updates. The system uses efficient data structures and heuristic algorithms to ensure scalability for networks containing up to 1,000,000 nodes.

5.2 System Architecture

The system consists of the following components:

1. **Input Module** – Loads network dataset (nodes, edges, weights, time).
2. **Graph Construction Module** – Converts input data into a graph using adjacency list representation.
3. **Node Importance Module** – Computes importance scores using centrality measures.
4. **Critical Node Detection Module** – Identifies critical nodes using greedy removal.
5. **Connectivity Evaluation Module** – Measures network connectivity using BFS.
6. **Temporal Update Module** – Updates edge weights for evolving networks.
7. **Output Module** – Generates list of critical nodes and connectivity metrics.

Module	Description	Technical Details
1. Input Module	Loads network dataset (nodes, edges, weights, time).	*Input format: CSV/JSON. Validates data integrity. Time complexity: O(n+m)*
2. Graph Construction Module	Converts input data into a graph using adjacency list representation.	*Data structure: Adjacency list. Space: O(n+m). Supports weighted/directed graphs.*
3. Node Importance Module	Computes importance scores using centrality measures.	*Algorithms: Betweenness, closeness, degree centrality. Time: $O(n^2)$ or O(nm) depending on method.*
4. Critical Node Detection Module	Identifies critical nodes using greedy removal.	*Greedy algorithm with node removal iteratively. Time: O(nm) for each iteration. Result: ranked critical nodes.*
5. Connectivity Evaluation Module	Measures network connectivity using BFS.	*Algorithm: Breadth-First Search (BFS). Time: O(n+m). Metrics: Connected components, largest component size.*
6. Temporal Update Module	Updates edge weights for evolving networks.	*Dynamic graph updates. Time window analysis. Incremental weight adjustments. Time: O(k) where k is edge updates.*
7. Output Module	Generates list of critical nodes and connectivity metrics.	*Output format: JSON/CSV. Includes rankings, scores, and network statistics. Export visualization-ready data.*

5.3 Logical Workflow

The system follows these steps:

Step 1: Data Input
Network data is loaded containing nodes, edges, and weights.

Step 2: Graph Construction
The network is represented as a graph:

$$G = (V, E, W)$$

using adjacency list for efficient storage.

Step 3: Node Importance Computation
Importance score of each node is calculated using centrality measures such as degree centrality and approximate betweenness centrality.

$$Score(v) = f(\text{centrality measures})$$

Step 4: Critical Node Identification
Nodes are ranked based on importance score. The node with the highest score is selected as a critical node and removed from the graph.

Step 5: Connectivity Evaluation
Breadth-First Search (BFS) is used to compute connectivity metrics such as largest connected component size.

Step 6: Temporal Weight Update
For temporal networks, edge weights are updated using adaptive weight function:

$$W_{new} = \alpha W_{old} + (1 - \alpha) W_{current}$$

Step 7: Iteration
Steps 3–6 are repeated until all critical nodes are identified.

Step 8: Output Generation
The system outputs the list of critical nodes and connectivity impact.

5.4 System Workflow Diagram (Conceptual)

Input Network Data

↓

Graph Construction

↓

Node Importance Computation

↓

Critical Node Detection

↓

Connectivity Evaluation

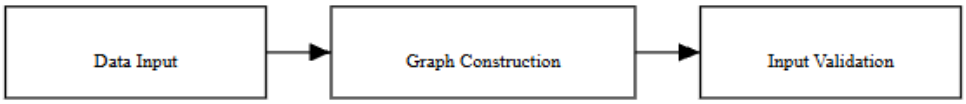
↓

Temporal Weight Update

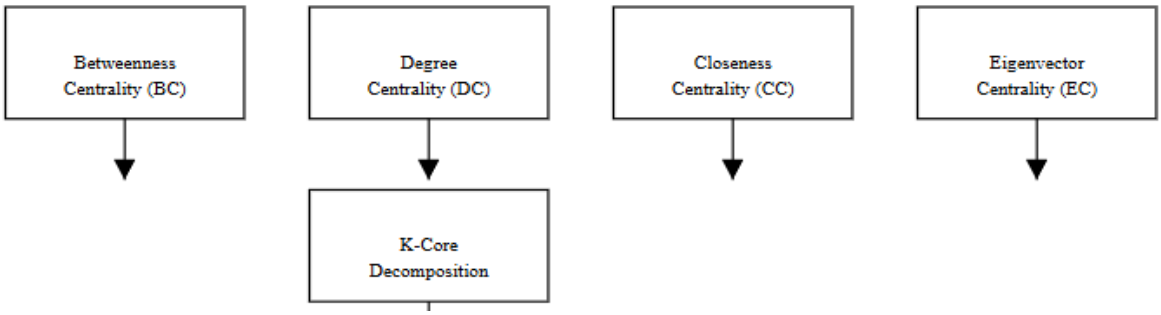
↓

Output Critical Nodes

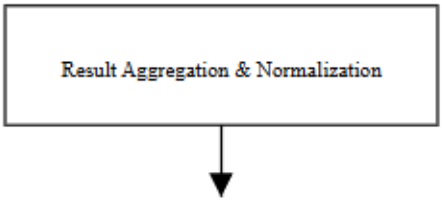
I. DATA ACQUISITION & GRAPH CONSTRUCTION



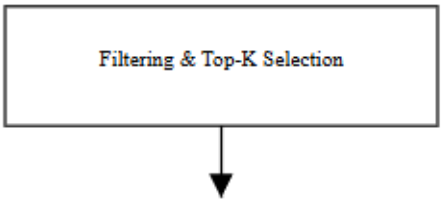
II. NODE IMPORTANCE COMPUTATION



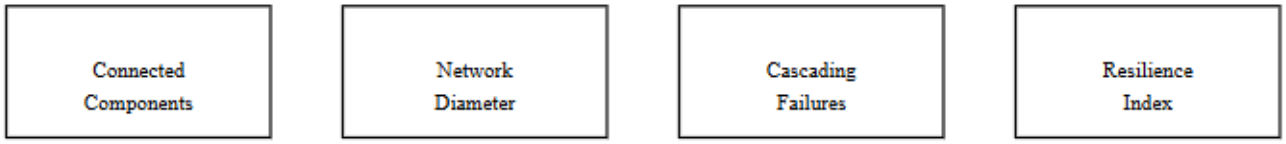
III. AGGREGATION & RANKING



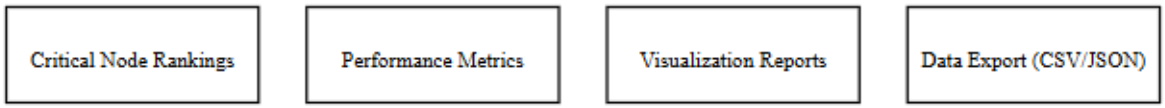
IV. CRITICAL NODE DETECTION



V. ROBUSTNESS & CONNECTIVITY EVALUATION



VI. OUTPUT GENERATION & ANALYSIS



5.5 Scalability and Efficiency

The system ensures scalability through:

- Adjacency list representation: Memory efficient
- Greedy heuristic algorithm: Fast and scalable
- Approximate centrality measures: Reduced computation time
- BFS traversal: Efficient connectivity analysis

Time complexity per iteration:

$$O(V + E)$$

The proposed methodology provides a scalable and efficient framework for critical node detection in large-scale and temporal networks. The system uses graph modeling, centrality computation, greedy node removal, connectivity analysis, and adaptive weight updates to accurately identify critical nodes while maintaining computational efficiency.

6. Technical Feasibility and Planning

6.1 Technical Feasibility

The proposed system is technically feasible as it utilizes efficient graph algorithms and scalable data structures suitable for large-scale networks containing 10^5 to 10^6 nodes. The adjacency list representation ensures memory efficiency with space complexity $O(V + E)$, making it suitable for large graphs. Heuristic and approximation algorithms such as degree centrality, approximate betweenness centrality, and greedy node removal reduce computational complexity and enable efficient execution.

Additionally, temporal network support is achieved through adaptive weight recalculation, allowing efficient updates without complete recomputation. These methods are supported by existing graph processing libraries, ensuring practical implementation.

6.2 Hardware Requirements

The system can be implemented using standard computing hardware.

Minimum requirements:

- Processor: Intel i5 or equivalent
- RAM: 8 GB (16 GB recommended for large datasets)
- Storage: 10 GB available disk space
- Platform: Windows, Linux, or cloud platforms such as Google Colab

Cloud platforms can be used to handle large-scale networks efficiently.

6.3 Software and Tools

The following software tools will be used:

Tool	Purpose
Python	Programming language
NetworkX	Graph construction and analysis
NumPy, Pandas	Data processing
Matplotlib	Visualization
SNAP / PyTorch Geometric	Large-scale graph processing
Jupyter Notebook / Google Colab	Development and testing

These tools provide efficient graph modeling and scalability.

6.4 Dataset Availability

Publicly available datasets will be used for implementation and evaluation.

Examples include:

- SNAP social network datasets
- Facebook network dataset
- CA-GrQc collaboration network dataset
- Synthetic large-scale graphs (Barabási–Albert model)

These datasets support scalability testing up to one million nodes.

6.5 Implementation Plan

The project will be implemented in the following steps:

1. Literature review and analysis of existing approaches
2. Dataset collection and graph construction
3. Implementation of centrality and greedy algorithms
4. Connectivity evaluation using BFS
5. Temporal weight update implementation

6. Performance evaluation and analysis

This step-wise approach ensures systematic development.

6.6 Time Schedule

Phase	Task	Duration
Phase 1	Literature review	1 week
Phase 2	Dataset preparation	1 week
Phase 3	Algorithm implementation	2 weeks
Phase 4	Temporal network implementation	1 week
Phase 5	Testing and evaluation	1 week
Phase 6	Documentation	1 week

Total estimated duration: 6–7 weeks.

6.7 Feasibility Conclusion

The proposed system is feasible in terms of computational requirements, tools, and datasets. The use of scalable algorithms and efficient graph representations ensures practical implementation for large-scale and temporal networks. The availability of suitable datasets and software tools supports successful execution within the planned timeframe.

7.Overall Conclusion

This project presents a scalable and efficient framework for detecting critical nodes in large-scale and temporal networks using graph-based algorithms. The study began with a detailed analysis of the Critical Node Detection Problem (CNDP), highlighting its importance in real-world systems such as communication networks, power grids, and social networks, where node failures can significantly impact connectivity and system performance.

A suitable graph abstraction model was developed using nodes, edges, and adaptive edge weights to represent both static and evolving networks. Scalable algorithms such as degree centrality, approximate betweenness centrality, greedy node removal, and connectivity analysis using Breadth-First Search (BFS) were selected and justified based on their efficiency and applicability to large networks. The methodology provided a systematic workflow, including graph construction, node importance computation, critical node identification, connectivity evaluation, and adaptive weight updates for temporal networks.

The literature survey demonstrated that while existing approaches provide effective solutions, many methods are limited in scalability and temporal adaptability. This project addresses these limitations by focusing on scalable algorithms capable of handling networks with up to one million nodes and supporting adaptive recalculation in evolving network environments.

The technical feasibility analysis confirmed that the proposed system can be implemented using available hardware, software tools, and publicly available datasets. The use of efficient data structures and approximation algorithms ensures practical implementation within reasonable computational constraints.

Overall, the proposed system provides a realistic, scalable, and efficient solution for critical node detection in large-scale and temporal networks. The framework contributes to improved network robustness analysis and provides a foundation for future research in dynamic network analysis and large-scale graph processing.